

Activity Recognition Using Hidden Markov Models

Oguzhan Oguz

oguzhano@kth.se

EQ2341 Pattern Recognition

Final project

- Section 1: recorded dataset
- Section 2: preprocessing and feature extraction
- Section 3: HMM model and Baum-Welch training
- Section 4: decoding and evaluation protocol
- Section 5: results, limitations, and conclusion

Project goal and recognition pipeline

Input signal

Phone accelerometer log

Each raw sample contains time, ax, ay, az.

Feature sequence

1 s overlapping windows

Each window is represented by a 3D magnitude-based feature vector.

Classifier output

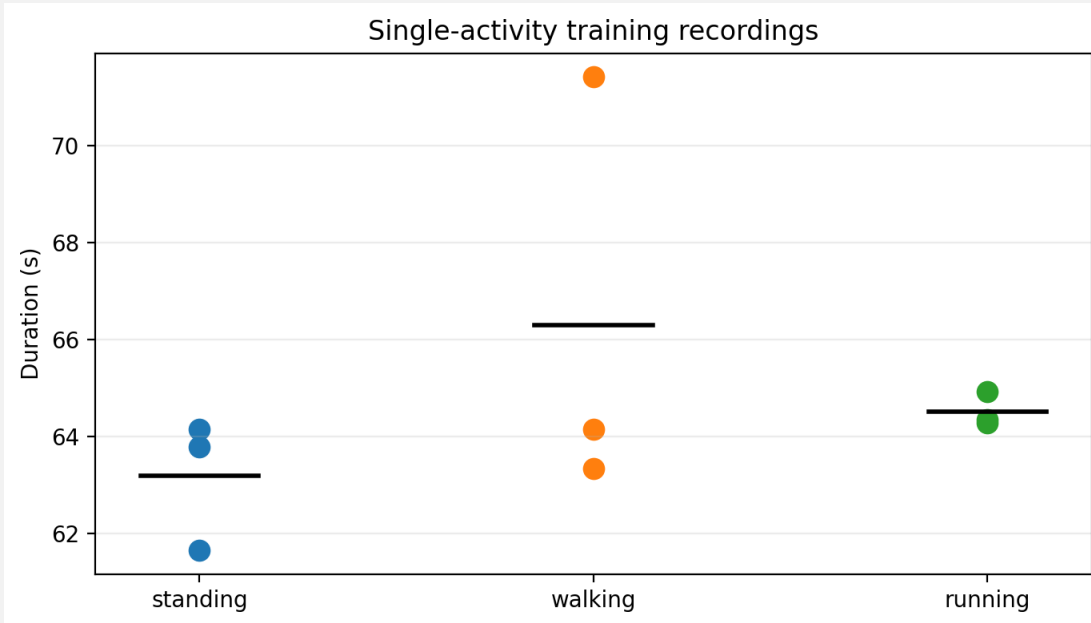
HMM state sequence

standing, walking, or running for each test window.

Why sequence classification?

Human activities are temporally persistent. The HMM uses this persistence through transition probabilities, instead of classifying each window independently.

Section 1 - Recorded dataset



Single-activity training recordings

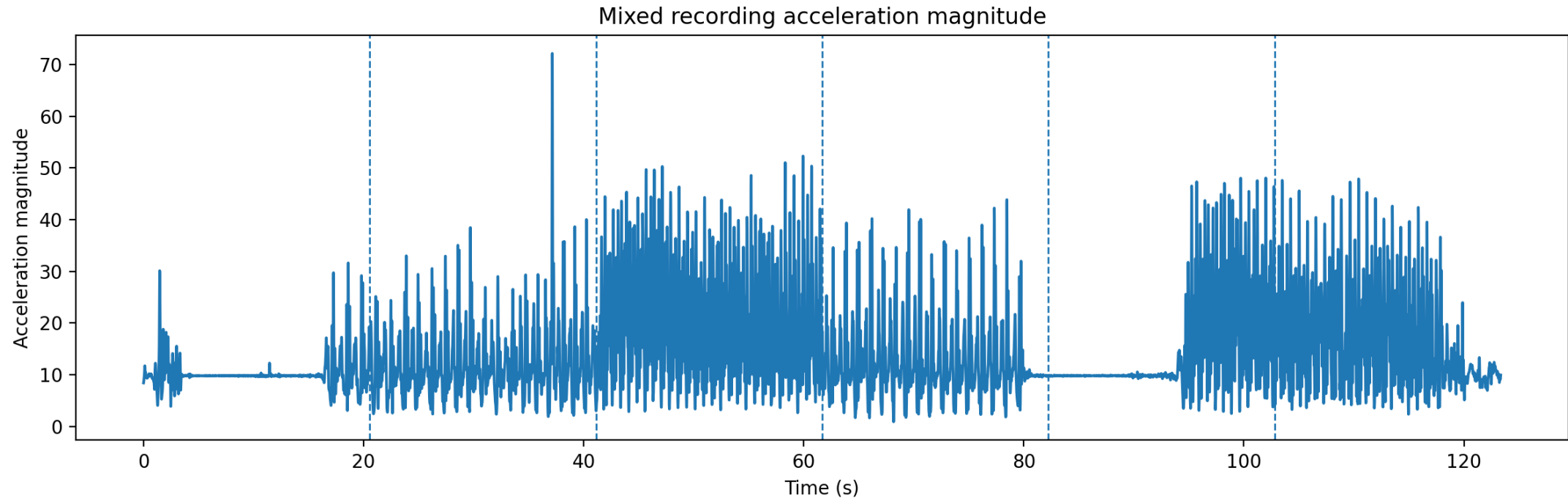
Recorded files

- 3 standing recordings (≈ 60 seconds)
- 3 walking recordings (≈ 60 seconds)
- 3 running recordings (≈ 60 seconds)
- 1 mixed test sequence (≈ 120 seconds)
- sampling rate: 125 Hz

Why separate training recordings?

Single-activity recordings provide clean initial labels for the emission distributions. The mixed recording is then used to evaluate the sequential classifier on unseen activity changes.

Section 1 - Raw mixed recording



Acceleration magnitude in the mixed test recording

The start, end, and activity-change regions may contain phone-handling artefacts. Therefore, the quantitative evaluation excludes **short temporal margins** around segment boundaries.

Section 2 - Preprocessing

Step	Purpose
Trim start/end	remove handling artefacts (first/last 8 s for training)
Magnitude	reduce sensitivity to phone orientation
Moving average	suppress short high-frequency fluctuations
Windowing	convert raw samples into a stable feature sequence

Chosen windowing

A **1 s window with 0.5 s hop** was used. This gives smoother trajectories, but neighbouring windows are overlapping and should not be interpreted as fully independent test samples.

Section 2 - Magnitude-based feature trade-off

Advantages

- less sensitive to phone orientation
- standing has low variation
- running has high variation
- compact enough for a small dataset

Trade-off

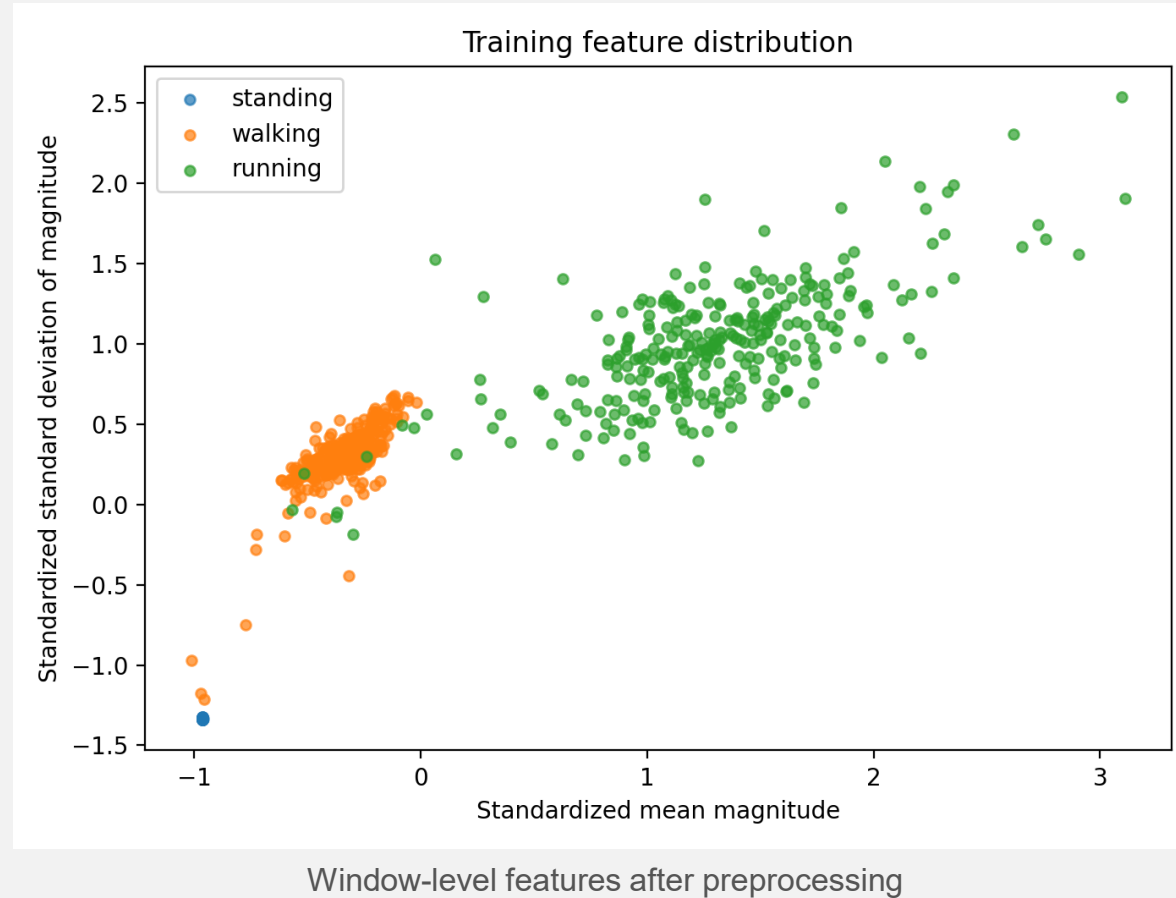
- directional motion is discarded
- transition artefacts can resemble running
- axis-specific patterns are not used
- richer features need more data

Feature vector

$$\mathbf{x}_n = [\text{mean}(m), \text{std}(m), \text{range}(m)]^T$$

Standardized using training-set statistics.

Section 2 - Training feature distribution



Standing is concentrated at low variation, running has the strongest magnitude variation, and walking lies between them. This supports a Gaussian emission model for each activity state.

Section 3 - HMM model structure

Hidden states

$S_t \in \{\text{standing, walking, running}\}$

The state at window t is the current activity.

Gaussian emissions

Each state uses a multivariate Gaussian density:

$$p(x_t \mid S_t = j) = \mathcal{N}(x_t; \mu_j, \Sigma_j).$$

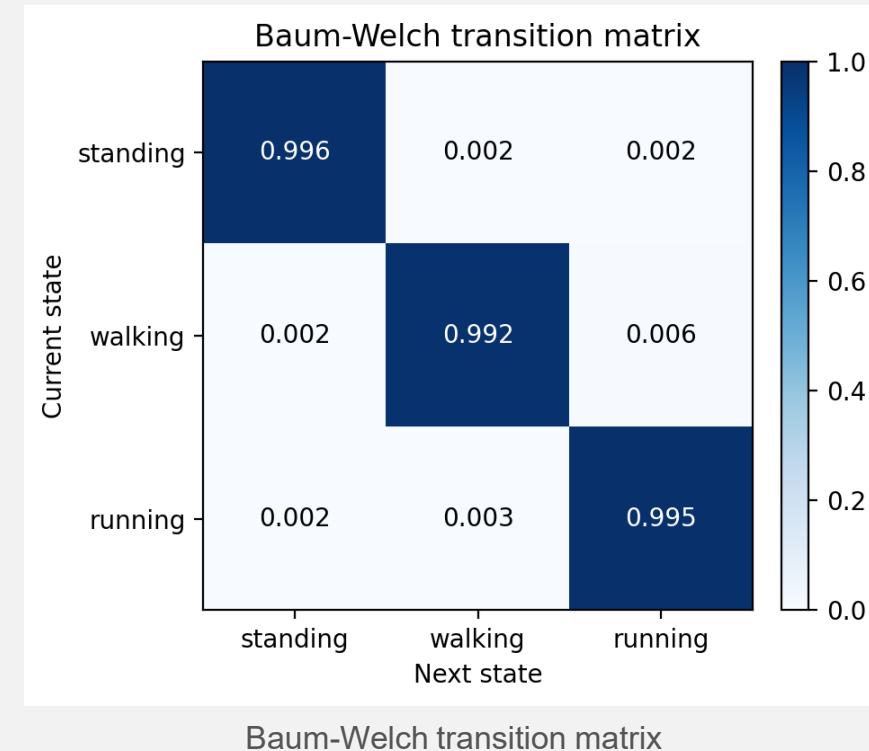
Why HMM?

Independent Gaussian decisions ignore **time persistence**. The HMM combines the **emission likelihoods** with a transition matrix, so short isolated errors are less likely to dominate the decoded activity sequence.

Section 3 - Initialization from labelled recordings

Initialization

- single-activity recordings provide stable initial labels
- Gaussian means and covariances are initialized per activity
- transition probabilities are initialized as mostly diagonal
- small pseudocounts keep all transitions possible



The transition matrix is mainly a persistence model. The large diagonal probabilities show that the HMM strongly favors temporal persistence.

Section 3 - Baum-Welch EM training

E-step

Forward-Backward computes **posterior state probabilities** $\gamma_j(t)$ and **transition posteriors** $\xi_{ij}(t)$.

M-step

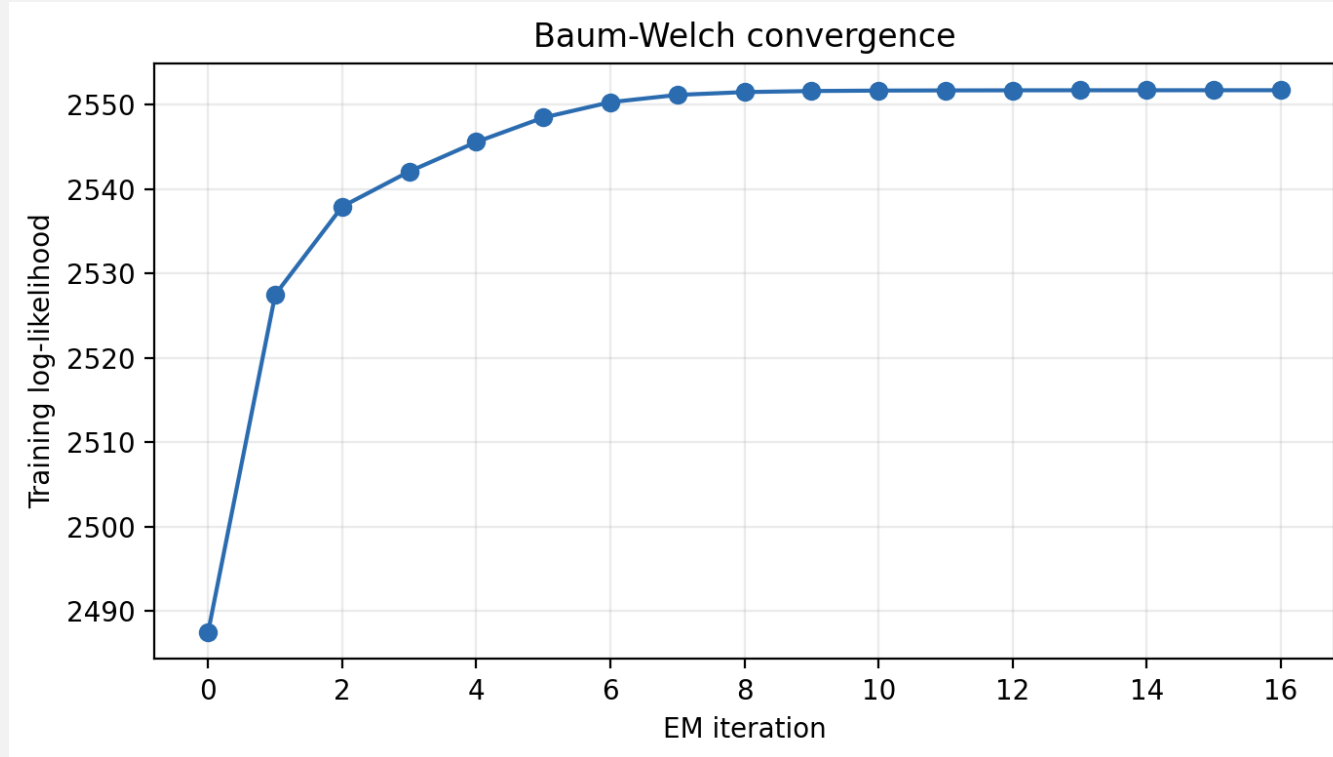
q , A , μ_j , and Σ_j are **updated** using the posterior weights from the E-step.

Training criterion

Stop when $|L(k) - L(k-1)| < 10^{-3}$.

The final run converged after 16 iterations.

Section 3 - Baum-Welch convergence



Training log-likelihood over EM iterations

Final values

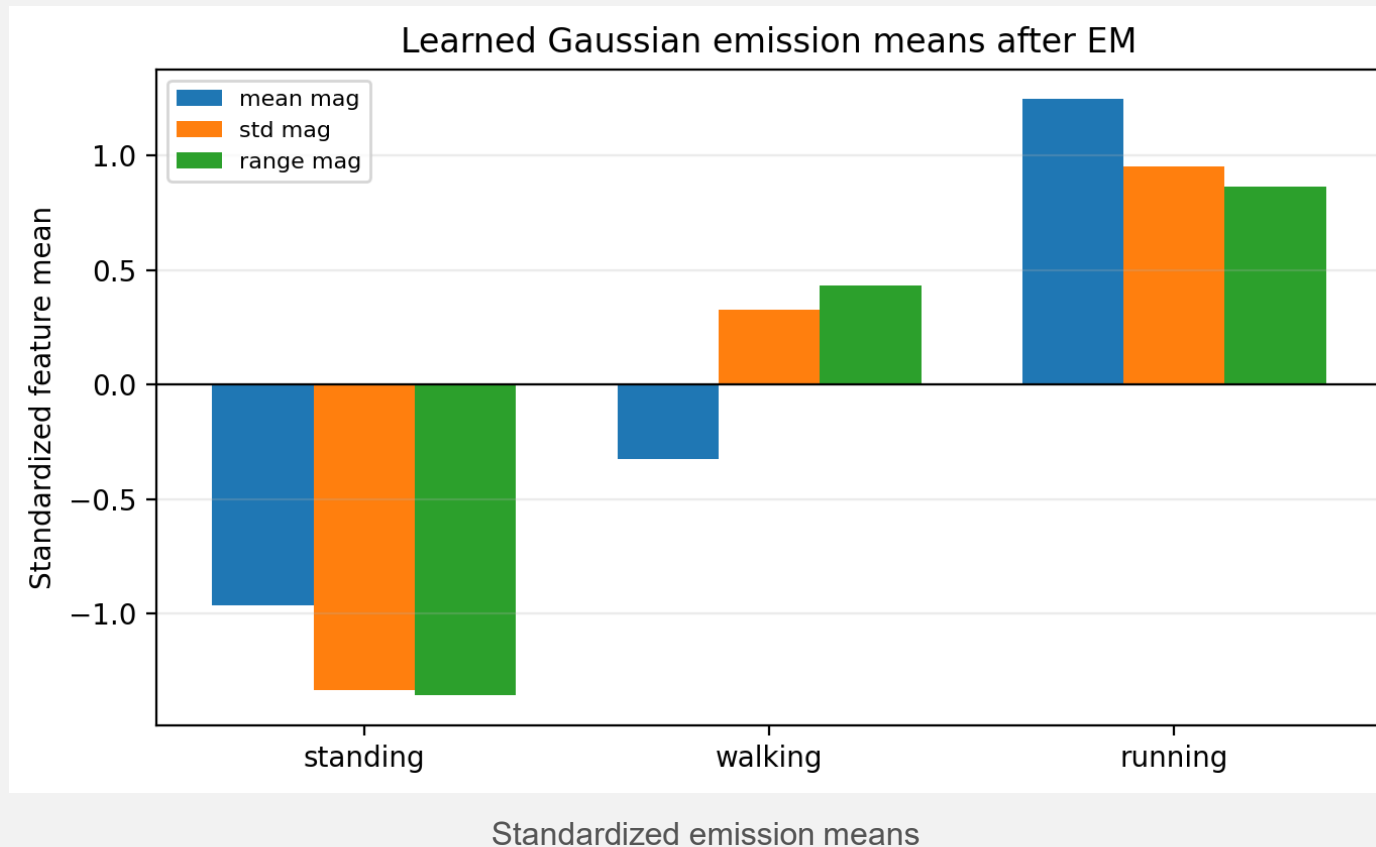
Initial log-likelihood:
2487.54

Final log-likelihood:
2551.70

Final improvement:
approximately 10^{-3}

The monotonic increase is the expected behaviour of Baum-Welch EM. The small final improvement indicates that the model has nearly converged for this dataset.

Section 3 - State identity after Baum-Welch



Identity check

- standing remains the lowest-variation state
- walking remains intermediate
- running remains the highest-variation state
- no state-label swap was observed

Why this matters

Baum-Welch is an unsupervised refinement step, so state labels are mathematically exchangeable. The **supervised initialization** kept the semantic state identities **stable**.

Section 4 - Decoding methods

Posterior MAP

Forward-Backward gives

$$\gamma_j(t) = P(S_t = j \mid X, \lambda).$$

The posterior MAP label is

$$\hat{S}_t = \arg \max_j \gamma_j(t).$$

Viterbi

Viterbi finds the most probable complete state sequence.

It usually gives smoother trajectories because it optimizes the **full path**.

Implementation check

Both decoders are reported because they answer related but different HMM inference questions.

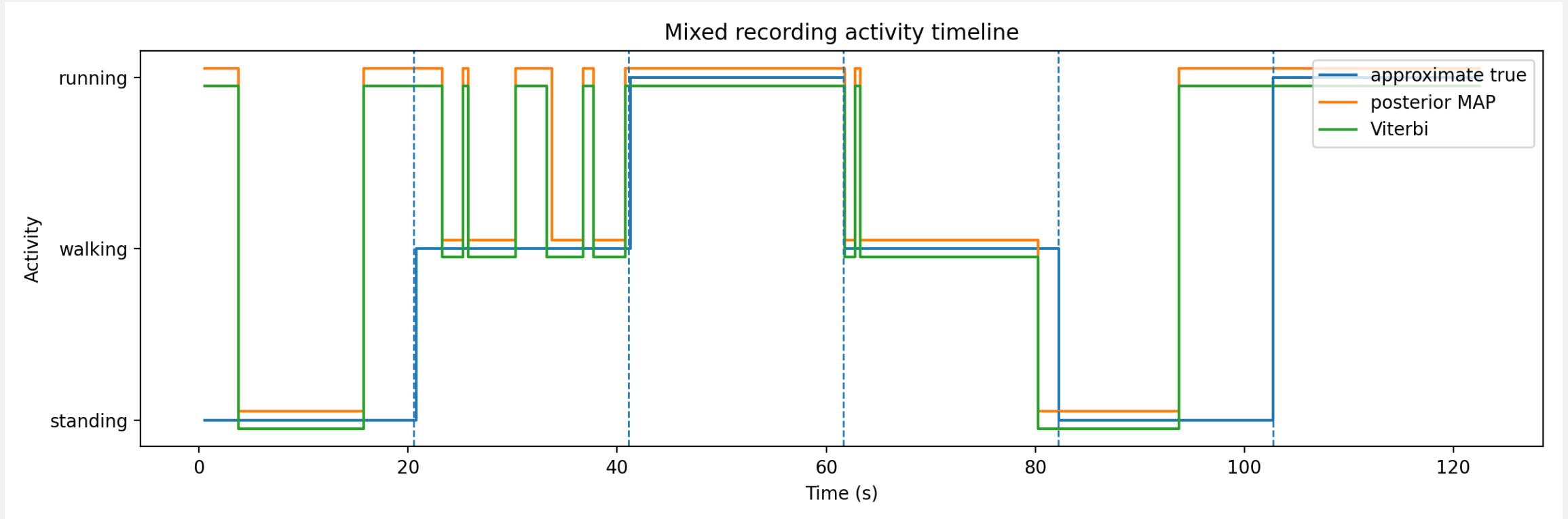
Section 4 - Evaluation protocol

Item	Choice used in this project
Training files	nine single-activity recordings
Test file	one mixed recording
Known order	standing -> walking -> running -> walking -> standing -> running
Boundaries	approx. equal-duration segments (exact timestamps unavailable)
Excluded margins	5 s at edges and 3 s around transitions
Metric	overlapping-window accuracy and confusion matrix

Important note

The reported accuracy is approximate because exact transition timestamps were not available. Some errors may therefore reflect label-boundary mismatch rather than only model mistakes.

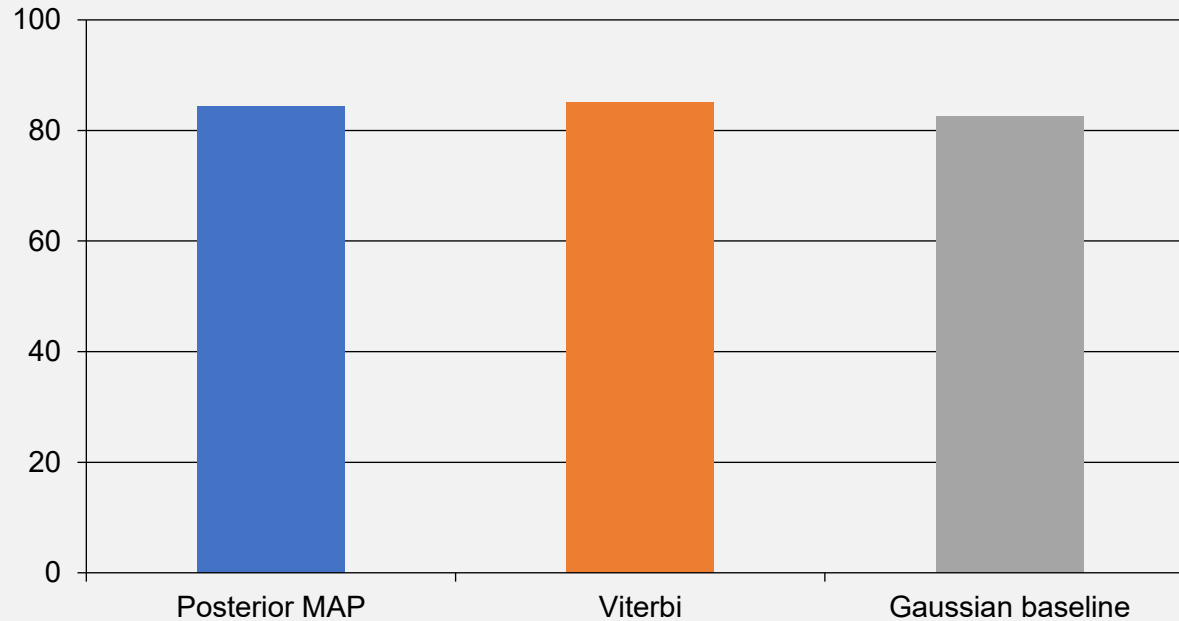
Section 4 - Mixed test sequence



The decoded trajectory follows the intended activity order, but some confusion appears near standing segments and transitions.

Section 5 - Accuracy and baseline comparison

Approximate accuracy comparison



Final numeric results

Posterior MAP: 84.4%

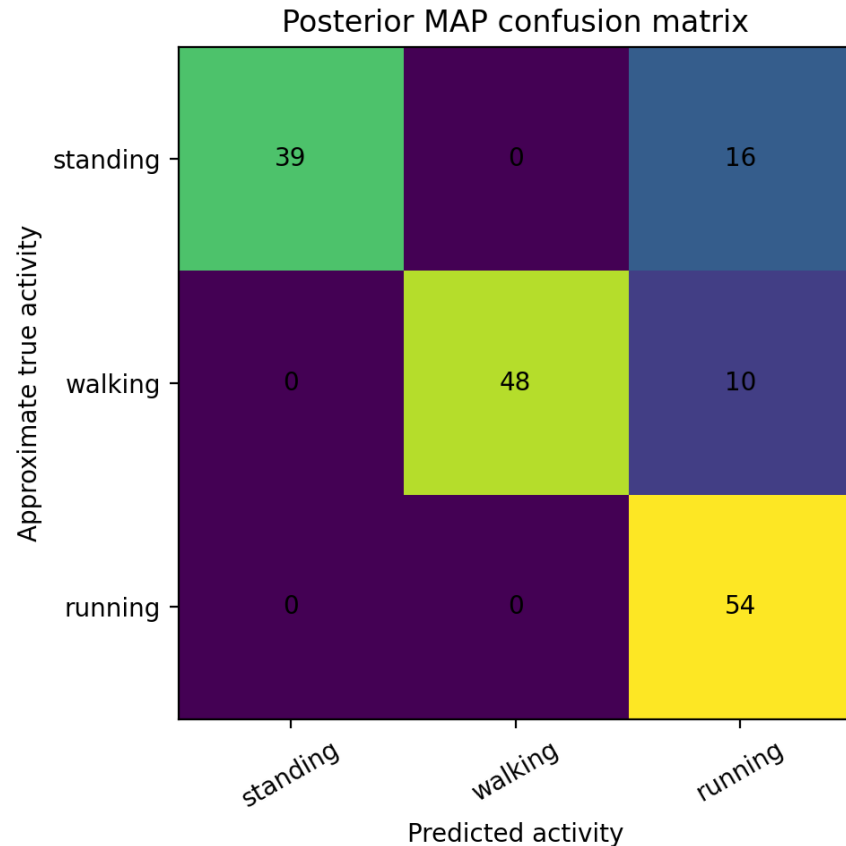
Viterbi: 85.0%

Independent Gaussian:
82.6%

Scored windows: 167

The HMM improves over the independent Gaussian baseline, mainly **by adding temporal persistence** through the transition matrix.

Section 5 - Confusion matrix



Main observations

- running: reliable
- walking: mostly correct
- main error: standing $\rightarrow$ running
- boundary uncertainty likely contributes

The standing-running confusion should not be interpreted only as an emission-model weakness; approximate transition labels may contribute.

Section 5 - Error analysis

Likely causes

- approximate equal-duration ground truth
- activity changes not exactly aligned with segment boundaries
- phone handling at start and end
- transient motion during standing segments

What worked well?

- window features separated the three activity levels
- Baum-Welch increased the likelihood
- Viterbi gave the best approximate accuracy
- HMM outperformed the non-temporal baseline

Final interpretation

The result is a working proof of concept: the classifier captures the activity sequence, while the remaining errors mostly occur where the reference labels are least certain.

Section 5 - Limitations

Dataset limitations

- only one user
- one phone placement
- three single-activity recordings per class
- one mixed test sequence
- no exact manual timestamps for transitions

Model limitations

- off-diagonal transitions are weakly informed
- overlapping windows are not independent
- magnitude features discard directional information
- Gaussian emissions are simple approximations

Future improvement

A stronger evaluation would use multiple mixed recordings, several users, exact transition timestamps, and possibly richer orientation-aware features.

Reproducibility package

Component	Included material
Code	executed Jupyter notebook
Data	raw sensor logs in data/raw/
Results	plots and CSV files in results/
Report	report PDF, LaTeX source, and presentation file
Instructions	README.md and requirements.txt

How to reproduce

Install the packages from requirements.txt and run the notebook from the project root. The notebook regenerates preprocessing, training, decoding, evaluation, and plots.

Conclusion

What was achieved?

- phone accelerometer data were recorded
- a 3-state Gaussian-emission HMM was trained
- Baum-Welch EM refined the model
- Forward-Backward and Viterbi decoding were evaluated

Final result

The best approximate mixed-test accuracy was 85.0% using Viterbi decoding.

The HMM outperformed the independent Gaussian baseline by using temporal persistence.

Main takeaway

The project demonstrates a complete and reproducible HMM-based activity recognition pipeline, while clearly stating the limitations caused by a small dataset and approximate mixed-test labels.

Thank you for your attention.

Questions are welcome.